

{% include header.html %}

{% include ai-share.html %}

作者：[TonTon Huang Ph.D.](#)

Blog：[2025年09月30日](#) · [Dify](#)、[Coze](#)、[n8n](#)、[AutoGen](#)、[LangChain](#)等熱門 Agent 框架

Dify、Coze、n8n、AutoGen、LangChain

快速梳理理解熱門 Agent 框架

最簡易上手與企業級可視化選 Dify/Coze/n8n；深度可編程、多智能體與可控狀態機選

LangChain/AutoGen/CrewAI。以下從架構、能力、限制、場景與選型決策全面梳理六大主流智能體框架。

框架速覽

- Dify
 - 架構與定位：開源 LLM 應用平台，內建工作流（Flow/DAG）、知識庫（RAG 全流程）、模型接入與觀測運維，面向企業快速落地與私有化部署。
 - 核心能力：可視化編排、工具/插件熱部署、資料接入與向量化、提示工程與版本管理、對話與任務類 App 模板、監控與配額。
 - 限制觀察：多-Agent 深度協作較弱，複雜任務圖需手動配置；進階狀態機與條件分支可讀性在大圖場景下降。
 - 適用場景：企業內知識助理、客服/營運自動化、多模態內容工具、RAG 系統 PoC→生產演進。
- Coze (扣子)
 - 架構與定位：零/低代碼的可視化 Agent 平台（SaaS 為主），提供 Bot Studio、工作流、插件與知識庫；底座編排技術以工程化 Orchestrator 為核心（如 Eino 等）。
 - 核心能力：拖拽流控、插件市場、多通道接入、排程（定時）與長期記憶、快速上線與運維看板。
 - 限制觀察：多-Agent 深度編排與可編程自由度不如專業框架；私有化與擴展需商業版本配合。
 - 適用場景：非技術人員快速原型、客服/行銷 Bot、多模態實用工具、企業內部流程輕自動化。
- n8n
 - 架構與定位：通用工作流自動化（源碼開放、可自託管），以節點式可視化為主，擅長對外部系統的廣泛整合。
 - 核心能力：400+ 應用連接器、低代碼 Function 節點、Webhook/事件驅動、AI/LLM 節點（模型/向量庫/工具可接），本地與雲端部署靈活。
 - 限制觀察：AI/Agent 功能偏「工作流中的一節」，非專門的多智能體協作引擎；大型任務圖可維護性需治理。
 - 適用場景：企業自動化中台、數據/CRM/工單整合、把 LLM 嵌入既有流程（如審核、摘要、比對）。

- AutoGen
 - 架構與定位：微軟開源多智能體對話編排框架，Python 為主，對話驅動 + 程式化控制並重。
 - 核心能力：Agent 間對話/協商、UserProxy (人類介入)、工具/函式呼叫、Agent 團隊 (GroupChat/Graph)、程式化終止條件、AutoGenBench 評測。
 - 限制觀察：主要 Python 生態；對開源模型/自建推理後端接入需工程整合；可視化與管理平台依賴周邊工具。
 - 適用場景：科研/高複雜協作、程式碼生成與調試、需要人機協作與嚴格控制流的企業任務。
- LangChain (含 LangGraph)
 - 架構與定位：模組化 LLM 應用開發框架 (Python/JS)，提供 Chains、Memory、Tools、RAG 模塊；LangGraph 提供顯式狀態機/圖 (可持久化)。
 - 核心能力：豐富連接器、Agent (ReAct/Plan&Execute 等)、LangSmith 觀測/評測、RAG 生產級拼裝、狀態圖 (檢查點、恢復、分支/合併)。
 - 限制觀察：學習曲線偏陡；單純用 Chains 易形成深層巢狀與調試困難，建議用 LangGraph 做清晰狀態管理。
 - 適用場景：工程化 RAG、可審計的多步推理/工具編排、需前後端雙語言支援與生態豐富的企業專案。
- CrewAI
 - 架構與定位：角色分工的多智能體協作框架 (Python)，以「團隊」與「任務」為核心抽象。
 - 核心能力：角色 (研究員/編輯/審稿等) 與流程 (順序/層級/並行)、任務拆解與交付、工具整合與長短期記憶插件。
 - 限制觀察：多模態與硬體/環境操控場景較弱；上手需閱讀原理與最佳實踐；運維可觀測較依賴外掛。
 - 適用場景：內容工廠 (調研→撰稿→審校)、多步資料分析、跨系統人工作業自動化替代。

核心能力橫向對比

維度	Diffy	Coze	n8n	AutoGen	LangChain/LangGraph	CrewAI
開發模式/語言	可視化 + 後端擴展 (Python 為主)	零/低代碼 (平台為主)	低代碼工作流 (Node 生態)	Python 可編程	Python/JS 模組化 + 狀態圖	Python 角色/任務導向
可視化程度	高	極高	極高	低 (社群工具可補)	中 (LangGraph 視覺化逐步加強)	中 (社群 Studio)
多-Agent 深度	中 (多工具/路由為主)	低-中 (多插件協作)	中 (多路分支可實現)	極高 (對話驅動/群聊)	高 (圖狀 agentic 模式)	高 (角色分工/流程)
工具/插件	市場 + 自定義工具	插件市場豐富	400+ 連接器	程式化 Tool/函式	大量連接器/MCP/工具包	工具需手動配置/套件

維度	Diffy	Coze	n8n	AutoGen	LangChain/LangGraph	CrewAI
模型/ 多模態	多雲/本地、常見多模態	多模態 插件完善	作為節點 調用模型	取決於集成	廣泛模型與多模態生態	取決於集成
狀態/ 記憶	對話/會話記憶、RAG、 工作流上下文	長期記憶、知識庫	工作流上下文/持久化	對話歷史、可插 記憶	Memory/Checkpoints/ 持久化	任務/團隊記憶 插件
觀測/ 調試	介面內建日誌/統計	運維看板	執行紀錄/ 重跑	AutoGenBench/ 追蹤	LangSmith/Tracing/Eval	日誌/ 可視化逐步完善
部署/ 私有化	易於私有化	SaaS 為主（企業版私有化）	自託管/雲端皆可	自託管	自託管（亦可雲）	自託管
生態與社群	高（企業導向）	高（平台導向）	高（自動化社群）	學研/高階工程	極高（連接器最豐）	中高（成長快）
代表強項	企業可視化 + RAG 全家桶	0→1 快速上線	系統整合王者	對話式多 Agent	工程化/狀態機/生態	角色分工式協作

多智能體協作模式

- 對話式協商（AutoGen）：Agent 以自然語言往復、協商、反思與驗證，易於人類介入與審計。
- 角色分工（CrewAI）：預設明確角色與交付，流程可順序/分層/並行，貼近真實團隊合作。
- 圖狀狀態機（LangGraph）：顯式節點/邊/Checkpoints，支援分支、重試、回退與持久化，最利於可觀測與可測試。
- 可視化工作流（Diffy/Coze/n8n）：以 DAG/節點圖實現工具串接與條件路由，快速落地但在跨回合推理/複雜記憶上需額外設計。

狀態、記憶與資料管理

- 短期記憶：會話上下文、任務中間結果（對話式協作需嚴控 token 與資料界限）。
- 長期記憶：向量庫/知識庫（分塊規則、重寫與檢索策略對效果影響大）。
- 任務狀態：顯式狀態機（LangGraph）或隱式對話狀態（AutoGen）；強建議對生產系統採用可持久化狀態。
- 工具輸入輸出治理：定義嚴格的結構化模式（JSON/Schema/GBNF），降低幻覺與注入風險。

安全與治理（企業/資安視角）

- 提示安全：輸入過濾、前置系統提示加固、上下文分區；對模型輸出做正則與結構校驗。
- 工具安全：設最小權限與作用域隔離（API Key、檔案系統沙箱、網路 Egress allowlist）；外呼 API 的重試/退避與速率限制。

- 程式執行：代碼工具必須沙箱（容器/Firejail/WASM）、資源/時間/網路配額、審計日誌。
- 秘密管理：集中保管（KMS/Secrets Manager）、短效憑證、落地前脫敏。
- 觀測與審計：全鏈路追蹤（請求→Agent 內部步驟→工具→外部系統）、可重放測試、資料最小化保留。
- 風險場景：多代理之間的反饋回路、供應鏈外掛、RAG 投毒、任務升權；採用策略引擎與政策即程式（Policy-as-Code）。

部署與整合建議

- 推理層：本地/雲端模型服務（vLLM、TGI、Ollama）統一為 OpenAI 兼容端點；多模型路由按任務/成本/延遲做策略。
- 向量層：企業偏好 PostgreSQL/pgvector；高吞吐可用 Milvus/Qdrant；定義離線重建與熱更新流程。
- 佈署形態：Dify/Coze 作前台 + 流程、LangGraph/AutoGen/CrewAI 為核心智能體引擎、n8n 作企業系統整合與排程。
- 可觀測：OpenTelemetry/Tracing + 指標（TTFT、TPOT、P95/P99、工具失敗率、上下文長度分布）；灰度與金絲雀。
- 成本控制：多池化（短上下文池/長上下文池）、推測解碼/草稿模型、回退策略（fallback）、向量重排避免冗長上下文。

選型決策指南（實操）

- 你要零/低代碼、最快原型與多通道上線 → 先用 Coze；若需自託管/企業資料在地化 → Dify 或 n8n。
- 你要把 AI 鑲嵌到既有業務系統與自動化 → n8n 為外層，內核選 Dify 或 LangChain。
- 你要可審計、可持久化的複雜狀態與多步推理 → LangGraph（LangChain 生態）。
- 你要深度多代理對話協作、人機共創與程式化控制 → AutoGen。
- 你要明確分工的內容/分析流水線（調研→撰寫→審校）→ CrewAI。
- 你要企業級 RAG 全家桶、快速到生產 → Dify（結合向量庫與私有模型服務）。
- 你要測試自動化/業務流程中台 → n8n + AutoGen 或 LangGraph 作內核 Agent。

典型參考架構

- 企業知識助理
 - 前台：Dify App（Chat/Toolflow）
 - 推理：vLLM 多模型路由（通用/長文/程式碼）
 - 向量：pgvector/Milvus，含資料治理與版本
 - 整合：n8n 連 CRM/工單、觸發通知
 - 觀測：LangSmith 或內建觀測，指標與追蹤打通
- 多代理程式碼助理
 - 內核：AutoGen 多代理（規劃/執行/測試/審核）
 - 工具：Repo/Issue/CI/CD/測試沙箱（容器隔離）
 - 門禁：人機協作 Gate（UserProxy）、變更審批
- 內容工廠
 - 內核：CrewAI（研究員→寫手→編輯→審校）
 - 工具：搜尋、RAG、事實核查、SEO 分析
 - 輸出：嚴格 JSON/Schema，過規則與增強校對

基準測試與驗證

- 工作負載：RAG 問答（短/長上下文）、多代理協作（分工/迭代/工具誤差）、結構化輸出（JSON/GBNF）。
- 指標：任務成功率、重試率、TTFT/TPOT、外部 API 失敗/退避、成本/千 token、審校工時節省。
- 方法：同模型/同溫度/同停止詞；固定資料切分與檢索器；用追蹤平台重放與差異比對。

與新興協議與趨勢

- MCP/A2A：多框架正增加對模型上下文協議與代理間協作協議的支援（以工具適配器或中介層實現），有助於標準化工具接入與跨平台組裝。
- 結構化輸出與約束編解碼：逐步成為生產必需（JSON 模式/語法約束/結構化評測）。
- 長期記憶與知識治理：企業從「能檢索」轉向「可治理/可追溯/可版本化」。